Report 1: Rudy, the reliable web server

Ilias Bourdakos

September 9, 2026

1 Introduction

This report aims to analyze and evaluate the behaviour of an implementation of a client-server communication paradigm, under multiple server and client configurations. Since the interaction of two or more machines is a topic that is heavily talked about in distributed systems, our findings can help create a knowledge baseline on how these systems operate.

Section 2 defines the implementations and infrastructure behind our experiments. Section 3 then analyzes the results, testing the server against same-machine and remote clients before turning to a multi-threaded implementation. Finally, Section 4 summarizes the key takeaways from our experiments.

2 Rudy

The mighty server that will be used in this analysis is Rudy. Rudy's initial implementation is a simple one. Just a single-threaded process that opens a *listening socket* based on a port and starts *accepting/receiving* client connections and requests. After a client request is sent to our server process, the server then parses it and then just sends a *response* back, using the **RFC2616** response standard. After that, the process is available again to handle the next request.

To use the server, a client first creates a connection using Rudy's IP and listening port, then sends data over the network (if remote) to the server process described earlier, formatted according to RFC2616 (as with the responses).The client then receives and reads the server's response. In a real environment, each request would typically require *server-side processing* before a response is sent, but here, we simulate this with a fixed 40ms delay before returning a constant string as the response.

3 Experiments & Evaluation

3.1 Same-Machine Client

The following tables present the time required (in microseconds), for a *single client* to issue 10, 100 and 1000 requests, while running on the same machine as Rudy. The request times were taken out of an average of 10.

Table 1 depicts the results when an artificial delay is present, while table 2 presents the results in the absence of any server-side delay.

Table 1: Benchmark results with artificial delay (40 milliseconds)

Requests	Time (μs)	Throughput
10	477040.8	20.962568
100	4707922.0	21.240794
1000	47168553.2	21.200565

Table 2: Benchmark results without artificial delay

Requests	Time (μs)	Throughput
10	20541.3	486.824106
100	189808.7	526.846240
1000	1864488.9	536.340013

Two expected findings: *i*) When a server-side delay is present, the request times increase while the *server throughput* decreases, and *ii*) Throughput stays pretty consistent within the same table, irregardless of the number of requests performed. Also, the fact that throughput fluctuates on table 2, indicates that the delay has a noticeable effect and it manages to *hide* any other small overhead that affected the request times.

Table 3: Comparison of time and throughput with & without artificial delay

Requests	Δ_{Time}	$\%\Delta_{Time}$	Δ_{TP}	$\%\Delta_{TP}$
10	456499.5	23.223496	-465.861538	0.043060
100	4518113.3	24.803510	-505.605447	0.040317
1000	45304064.3	25.298382	-515.139447	0.039528

Even though the time and throughput difference caused by the delay rises with more requests (something we expect, since the number of requests affected by the delay, increases), the % differences stay consistent. This means there isn't any performance degradation as the demand increases.

3.2 Remote-Machine Client

At this section, we examine the case where the client runs on a remote machine, but on the same Local Area Network (LAN). Table 4 shows the results of a remote-machine client, compared to the same-machine client we examined earlier.

Table 4: Results of same- and remote-machine performance (w/ delay)

Requests	Time (remote)	Δ_{Time}	$\%\Delta_{Time}$	Throughput	Δ_{TP}	$\%\Delta_{TP}$
10	730263.1	253222.3	1.530819	13.693695	-7.268873	0.653245
100	7750735.8	3042813.8	1.646318	12.902001	-8.338793	0.607416
1000	72110652.2	24942099.0	1.528787	13.867577	-7.332989	0.654114

As we can see, when the client is in a remote machine, the requests require greater times to be completed (about 1.5% more), and throughput drops. This can be explained as an extra network overhead, as each request travels outside of its source machine.

Table 5: Results of same- and remote-machine performance (no artificial delay)

Requests	Time (remote)	Δ_{Time}	$\%\Delta_{Time}$	Throughput	Δ_{TP}	$\%\Delta_{TP}$
10	117571.6	97030.3	5.723669	85.054554	-401.769552	0.174713
100	1231805.2	1041996.5	6.489719	81.181667	-445.664573	0.154090
1000	13274045.8	11409556.9	7.119402	75.334982	-461.005031	0.140461

This time, the difference is far more significant: the average gap between same- and remote-machine performance jumped from 1.5x to 5–7x in average request time (with a similar pattern in throughput). This again shows that the artificial delay is **significant enough** to mask the overhead introduced by sending requests from a different machine.

3.3 Multi-threaded Rudy

To avoid lengthening this analysis, we'll fix some experiment variables. First, a multi-threaded server only makes sense to test with a server-side processing delay: without one, responses return fast and the same thread can handle the next request with little bottleneck. Multi-threading shines when each request requires processing time, since a busy thread can be replaced by another. We'll also limit testing to **100 requests per client** (not 10 or 1000). Our case of interest is when Rudy receives a new request before finishing the previous one. This can't be simulated with a single client process, since it waits for the server to respond before sending the next request. So Rudy needs to be hit by multiple clients simultaneously (oh no).

3.3.1 Same-Machine

This section covers the case of multiple concurrent clients in the same machine as our server.

Table 6: Results by clients and server threads (100 requests + delay)

Clients	Server Threads	Time	Throughput	Δ_{Time}	$\%\Delta_{Time}$
2	4	4744910.700	42.150424	-	-
4	4	4833308.250	82.759050	-	-
6	4	7066947.900	84.902282	2359025.900	1.501076
4	6	4742813.125	84.338132	-	-
6	6	4745018.900	126.448390	-	-
8	6	6282988.725	127.327938	1575066.725	1.334557
6	8	4910567.400	122.185473	-	-
8	8	4949041.600	161.647459	-	-
10	8	5961453.720	167.744320	1253531.720	1.266260

Each concurrent client sends 100 requests, repeated 10 times (to average over 10 runs). With multiple clients, table values represent the average across all of them. Comparing Table 6 to the single-client, same-machine baseline with artificial delay (4707922.0): in the first two rows, when server threads >= clients, average request time matches the single-client case, even with 2 or 4 concurrent clients. Meanwhile, throughput doubled with 2 clients and quadrupled with 4 (baseline: 21.240794). This suggests each client was served by a separate thread with no bottlenecks, letting the server double and quadruple its throughput and scale cleanly with more users. The same pattern holds for the other rows as well (4–6 clients on 6 threads, 6–8 clients on 8 threads). In the other cases, when we have more concurrent clients than server threads, the

average request times increase, since now some clients will have to wait for a server-thread to become available. Simultaneously, the throughput reaches a maximum, something that makes sense since we surpassed the number of clients we can satisfy at the same time. Indeed, when concurrent clients exceed available threads by 2, the average request time is roughly 1.4x larger (in microseconds).

While we don't have enough data to draw a firm conclusion, the gap in request times also seems to shrink as clients and threads scale up together ($1.5 \rightarrow 1.3 \rightarrow 1.2$). This may suggest that a production server with hundreds of threads could handle thousands of concurrent users without significant bottlenecks.

3.3.2 Remote-Machine

This section covers the case where the multiple concurrent clients are in a remote machine in the same LAN.

Table 7: Results by clients and server threads: same- vs remote-machine

Clients	Server Threads	$Time_{same}$	$Time_{remote}$	TP_{same}	TP_{remote}	Δ_{Time}	$\%\Delta_{Time}$	Δ_{TP}	$\%\Delta_{TP}$
2	4	4744910.700	6506423.400	42.150424	30.738854	1761512.700	1.371243	-11.411570	0.729266
4	4	4833308.250	7663298.775	82.759050	52.196843	2829990.525	1.585518	-30.562208	0.630709
6	4	7066947.900	7620268.350	84.902282	78.737385	553320.450	1.078297	-6.164897	0.927388
4	6	4742813.125	7388130.000	84.338132	54.140899	2645316.875	1.557753	-30.197233	0.641950
6	6	4745018.900	7369110.800	126.448390	81.420950	2624091.900	1.553020	-45.027440	0.643907
8	6	6282988.725	7875873.425	127.327938	101.576036	1592884.700	1.253523	-25.751903	0.797751
6	8	4910567.400	7873652.300	122.185473	76.203517	2963084.900	1.603410	-45.981956	0.623671
8	8	4949041.600	6913271.100	161.647459	115.719460	1964229.500	1.396891	-45.927999	0.715876
10	8	5961453.720	6352645.000	167.744320	157.414746	391191.280	1.065620	-10.329574	0.938421

In the case of the same-machine multi-threaded approach, we saw that when the server threads were less than the clients, the increase in the average requests time was noticeable. When the clients are remote, though, we see in table 7 that when they surpass the number of server-threads, the time growth is not that substantial (they diverge closer to same-machine times). The same is true for the throughput difference. **This suggests the networking overhead from remote requests is being overtaken by the thread-availability bottleneck**.

Table 8: Multi-threaded vs. single-threaded performance differences

Clients	Server Threads	Δ_{time}	$\%\Delta_{time}$	Δ_{TP}	$\%\Delta_{time}$
2	4	1244312.400	1.191244	-17.836853	0.419729
4	4	87437.025	1.011410	-39.294842	0.247180
6	4	130467.450	1.017121	-65.835384	0.163861
4	6	362605.800	1.049080	-41.238898	0.238304
6	6	381625.000	1.051787	-68.518949	0.158460
8	6	-125137.625	0.984111	-88.674035	0.127018
6	8	-122916.500	0.984389	-63.301517	0.169310
8	8	837464.700	1.121139	-102.817459	0.111494
10	8	1398090.800	1.220080	-144.512745	0.081962

The last four columns of Table 8 use the single-threaded remote baseline (100 requests+artificial delay: 7750735.8 time, 12.902001 throughput). The table shows that multiple threads let us serve up to 10 remote concurrent clients with roughly the same average request time as a single client, as $\%\Delta_{time}$ is near 1 on ever row (values bellow 1 indicate multi-client requests handled slower). The throughput columns confirm the server handles greater workloads than the single-threaded approach (negatives in the third column indicate a throughput increase).

4 Conclusions

To summarize, this report covered the testing and evaluation of *Rudy*, our trusty web server. Through the findings of the experiments, we produced information regarding networking overhead and how it affects request times and server throughput, we analyzed the benefits of using a multi-threaded server process and made many comparisons throughout the way. More specifically, we proved that network and IO overhead definitely have an effect in the communication speed, but it quickly gets overtaken by factors like server-side delay and unavailable server-threads. This project has been a great practical exercise to understand these topics that are indeed very important in the distributed systems field.

Data Availability

All scripts, raw data, and analysis notebooks used in this report are available as extra provided files accompanying this report.